



UNIVERSIDAD NACIONAL DE CÓRDOBA
FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y
NATURALES

SÍNTESIS DE REDES ACTIVAS

Tarea Complementaria N°2
Diseño de Filtros Activos con Python

Profesor Titular: Dr. Ing. Ferreyra Pablo

Profesor Adjunto: Ing. Reale Cesar

Alumnos: Dávila Tomassi, Carlos Valentino
Mendes Rosa, Agustín
Monja, Ernesto Joaquín

Año 2025

Índice

1. Introducción	2
1.1. Consigna	2
2. Desarrollo	2
2.1. Introducción a los Filtros Activos	2
2.2. Python como Software de Diseño	3
2.2.1. Obtención de Funciones de Transferencia a través de Especificaciones	3
2.2.2. Representación de la Función de Transferencia en Bicuadráticas . .	6
2.2.3. Gráficas en Python	9
3. Conclusión	13
4. Bibliografía	13
5. Anexo	13

1. Introducción

1.1. Consigna

En este presente informe, se detalla de forma conceptual el funcionamiento de las distintas librerías que cuenta el programa Python para poder sintetizar filtros activos en base a ciertos requerimientos que le presente el programador tal así como se realizo para el Trabajo Práctico N°4 de esta asignatura.

2. Desarrollo

2.1. Introducción a los Filtros Activos

De acuerdo a la bibliografía utilizada, se define a un filtro como una red utilizada para cambiar la forma del espectro de la frecuencia de una señal eléctrica y estos son clasificados de acuerdo a la función que realizan, es decir que forma le dan al espectro de frecuencias.

Como se menciona anteriormente, los filtros se definen de acuerdo a que forma le dan al espectro de las frecuencias lo cual implica un uso de las Transformadas de Laplace y consecuentemente el análisis de funciones de transferencias. De acuerdo a la forma de la función de transferencia que tenga el filtro, es decir sus ceros, polos y constantes, se definirá la función del filtro tal como se presenta en la siguiente figura:

Filter function	Transfer function
Low-pass	$K \frac{1}{s^2 + as + b}$
High-pass	$K \frac{s^2}{s^2 + as + b}$
Band-pass	$K \frac{s}{s^2 + as + b}$
Band-reject	$K \frac{s^2 + d}{s^2 + as + b}$
Delay equalizer	$\frac{s^2 - as + b}{s^2 + as + b}$

Figura 1: Funciones de Transferencia de distintos tipos de filtros.

La bibliografía utilizada menciona como sintetizar las funciones de transferencia de cualquier tipo de filtro en componentes que, mediante una topología activa del filtro, definirán los elementos circuitales a utilizar, sin embargo no se realiza un análisis de como

deducir las funciones de transferencia de acuerdo a las especificaciones de un filtro (por ejemplo, frecuencias de corte, atenuaciones máximas y mínimas, etc.).

Se busca entonces en este informe, detallar el proceso de como realizar funciones de transferencia de acuerdo a las especificaciones de un filtro utilizando librerías de Python.

2.2. Python como Software de Diseño

La elección de Python como software de diseño para los filtros se debe a la ventaja que este posee al ser Software Gratuito y de Código Abierto, permite entender los algoritmos que ejecutan los comandos de sus librerías además de ser un lenguaje de programación de uso general lo que no nos limita únicamente a fines matemáticos (como si lo hace Matlab/Octave) o fines electrónicos (como PSpice/LTSpice).

2.2.1. Obtención de Funciones de Transferencia a través de Especificaciones

La librería *SciPy* de Python ofrece, entre tantas funciones, el módulo *.signal* el cual se dedica al análisis y procesamiento de señales y sistemas lineales. Este módulo carece de los conocimientos de electrónica utilizados para sintetizar filtros con topologías Sallen-Key por ejemplo, pero si sabe determinar de forma matemática las funciones de transferencia que representan los requerimientos de un filtro, es decir por ejemplo:

- La cantidad de polos que necesita un filtro Butterworth
- La ubicación del Ripple (Banda de Paso o de Rechazo) de un filtro Chebyshev
- La aproximación deseada para un filtro.
- Si el filtro será digital o analógico.

Entre otras funciones. Para la obtención de las funciones de transferencia, este módulo cuenta con 5 funciones que nos entregan distintos tipos de aproximaciones para los filtros, las cuales se las presenta en la siguiente tabla:

Función	Aproximación	Característica Principal de la Aproximación
<i>signal.butter()</i>	Butterworth	Máxima Planicidad en la Banda de Paso.
<i>signal.cheby1()</i>	Chebyshev I	El ripple del Chebyshev se encuentra en la Banda de Paso.
<i>signal.cheby2()</i>	Chebyshev II	El ripple del Chebyshev se encuentra en la Banda de Rechazo.
<i>signal.ellip()</i>	Elíptica	Hay ripple en ambas bandas, pero entre ellas hay una transición mas abrupta.
<i>signal.bessel()</i>	Bessel	Se obtiene Fase casi lineal en la Banda de Paso.

Cuadro 1: Funciones principales del módulo *SciPy*.

Las funciones presentadas en la Tabla 1, surgen todas y cada una de ellas de una función más general denominada *iirfilter()*, la cual además de tomar los parámetros que se listarán a continuación, recibe una especificación de que aproximación se utiliza para el filtro (Véase en el Anexo 1: Fragmento de la librería *SciPy*, donde se muestra la función *iirfilter()*). Veamos entonces los parámetros típicos que toma esta función como entrada:

- n: Orden del filtro pasa bajos normalizado. Se tiene que la librería *SciPy*, genera filtros pasa bajos normalizados los cuales luego mediante transformaciones de la variable s , los transforma a otro tipo de filtros tal como se presenta en la siguiente tabla:

Tipo de filtro	Transformación en s	Parámetros	Orden final
Pasa bajos (LP)	$s \rightarrow \frac{s}{\omega_c}$	ω_c : frecuencia de corte	N
Pasa altos (HP)	$s \rightarrow \frac{\omega_c}{s}$	ω_c : frecuencia de corte	N
Pasa banda (BP)	$s \rightarrow \frac{s^2 + \omega_0^2}{BW s}$	$\omega_0 = \sqrt{\omega_1 \omega_2}$	$2N$
Rechaza banda (BS)	$s \rightarrow \frac{BW s}{s^2 + \omega_0^2}$	$\omega_0 = \sqrt{\omega_1 \omega_2}$	$2N$

Cuadro 2: Transformaciones de frecuencia aplicadas al prototipo pasa bajos de orden N .

En la última columna, se define al orden final del filtro, el cual nos indica que si bien se puede indicar que el filtro sea de orden N , dada la transformación utilizada, el orden del mismo puede aumentar a $2N$. Por último, se define a $BW = \omega_2 - \omega_1$ siendo este el ancho de banda angular del filtro pasa banda o rechaza banda.

- rp: Ripple en $[dB]$ para la banda de paso o la banda de rechazo si se trata de un filtro pasa banda o rechaza banda respectivamente.
- wp: Frecuencia crítica normalizada de los polos.
- btype: Tipo de filtro, este puede ser:
 - `btype = 'lowpass'`: Filtro pasa bajos.
 - `btype = 'highpass'`: Filtro pasa altos.
 - `btype = 'bandpass'`: Filtro pasa banda.
 - `btype = 'bandstop'`: Filtro rechaza banda.
- analog: Indica si se busca diseñar un filtro en tiempo continuo o discreto mediante una variable booleana (es decir, *'true'* o *'false'*). Este parámetro es de vital importancia ya que no es lo mismo diseñar un filtro analógico, es decir en tiempo continuo que en tiempo discreto ya que la herramienta de análisis y traspaso al dominio de la frecuencia cambia. Esto quiere decir que si en tiempo continuo se utiliza la Transformada de Laplace, para tiempo discreto se utiliza la Transformada Z.

- output: Se trata de un parámetro opcional el cual me indica el tipo de salida de la definición. El parámetro de salida puede ser:

output	Descripción
'ba' o 'tf'	Coefficientes de la Función de Transferencia
'zpk'	Cero, Polo y Constante
'ss'	Representación en Variables de Estado
'sos'	Representación en Bicuadráticas

Cuadro 3: Valores del parámetro 'output'

- fs: Se define aquí la frecuencia de sampleo utilizada en caso de que se este diseñando un filtro de tipo digital. Si el filtro es analógico, este parámetro no es necesario utilizarlo.

Para aplicar estos conceptos, veamos el siguiente ejemplo, donde se utiliza la aproximación de Chebyshev I, para diseñar un filtro rechaza banda, en un rango de frecuencias de 1000 [Hz] a 2000 [Hz]:

```

1  import numpy as np
2  from scipy import signal
3
4  ## Ancho de Banda a Rechazar:
5  f1 = 1000                                # Frecuencia Inferior: 1000 [Hz]
6  f2 = 2000                                # Frecuencia Superior: 2000 [Hz]
7  Wbw = [2*np.pi*f1, 2*np.pi*f2]
8
9  ## Aproximación de un filtro rechaza banda, analógico y de segundo orden, con un
10 ## ripple en la banda de rechazo de 1 [dB] que sigue las frecuencias listadas
11 ## anteriormente.
12 num, den = signal.cheby1(N = 2,           # Filtro de 2do Orden
13                           rp = 1,         # Ripple de 1 [dB]
14                           Wn = Wbw,       # Frecuencias a Rechazar
15                           btype = 'bandstop', # Filtro rechaza banda
16                           analog = True,  # Filtro analógico
17                           output = 'ba')  # Salida del tipo: F(s) = num/den
18
19 ## Impresión de los parámetros obtenidos:
20 print("Numerador:", num)
21 print("Denominador:", den)

```

Al compilar este código, obtendremos en la terminal el siguiente mensaje impreso:

```

Numerador: [8.91250938e-01 0.00000000e+00 1.40740707e+08 0.00000000e+00
5.55622040e+15]
Denominador: [1.00000000e+00 6.25596698e+03 1.93721424e+08 4.93951354e+11
6.23418183e+15]

```

Figura 2: Salida por consola del Ejemplo 1.

Observamos en este ejemplo una peculiar característica de la forma de trabajar que tiene esta librería; donde si bien se definió a un filtro rechaza banda de 2^{do} orden, al compilar el código se obtiene una función de transferencia de 4^{to} orden dado su denominador.

Esto se debe a que la librería de Python *SciPy* siempre parte de un prototipo de pasa bajos el cual mediante transformaciones *HP*, *BP* ó *BS* (High-Pass, Band-Pass y Band-Stop del Inglés) lo transforma en el filtro deseado de acuerdo a la Tabla 2. Justamente en este ejemplo, la transformación utilizada es la del rechaza banda, la cual duplica el orden del filtro original, ya que cada polo del prototipo pasa bajos se transforma en un par de polos complejos conjugados en el rechaza banda; y lo mismo es de esperarse si se utiliza un filtro pasa banda de las mismas características.

Si bien hemos obtenido una función de transferencia, no se trata de las formas bicuadráticas que se utilizan en la bibliografía para sintetizar filtro de forma activa, por lo tanto se requiere un paso adicional para obtener funciones de transferencia de 2^{do} orden y poder realizar los filtros activos.

2.2.2. Representación de la Función de Transferencia en Bicuadráticas

Los parámetros que nos entregan las funciones descritas en el apartado anterior son el numerador y denominador de la función de transferencia global del filtro y suelen ser de la forma:

$$T(s) = \frac{b_ms^m + b_{m-1}s^{m-1} + \dots + b_2s^2 + b_1s + b_0}{a_ns^n + a_{n-1}s^{n-1} + \dots + a_2s^2 + a_1s + a_0} \quad (1)$$

Sin embargo se tiene que según la Figura 1, un filtro no puede ser de orden mayor a 2 tanto en el denominador debido a que se llegan a los límites de estabilidad del amplificador, tal así como el numerador no puede exceder el grado del denominador por cuestiones de realizabilidad de redes. Por lo tanto debemos hacer uso de la descomposición en bicuadráticas y para ello haremos uso de la función *signal.tf2sos()* que nos ofrece *SciPy*.

Esta función internamente separa la función de transferencia mostrada en (1) en distintas bicuadráticas. Para ello primeramente se hace uso de forma interna de la función *signal.tf2zpk()* la cual descompone a la función de transferencia entre sus ceros, polos y constante para luego agruparlos de forma conveniente de acuerdo a su cercanía al círculo unitario. Se incluye en este informe (Véase el Anexo 2) un fragmento del código de la librería de *SciPy*, el cual explica como funciona este algoritmo.

Al ejecutar esta función, obtendremos una función de transferencia del siguiente estilo:

$$T_i(s) = \frac{b_{0,i}s^2 + b_{1,i}s + b_{2,i}}{s^2 + a_{1,i}s + a_{2,i}} \quad (2)$$

De modo tal que se puede llegar a la función original mediante una productoria de todas las $T_i(s)$, es decir:

$$T(s) = \prod_{i=1}^{\infty} T_i(s) \quad (3)$$

Al compilar esta función devuelve un array de 6 elementos, los cuales representan los coeficientes de la función de transferencia descrita en (2). Nótese que *SciPy* siempre normaliza el termino $a_0 = 1$, por lo que debe ser tenido en cuenta cuando se obtienen los coeficientes de las bicuadráticas.

Veamos entonces, una aplicación de esta función al Ejemplo 1, donde se diseñó un filtro rechaza banda, con una banda de rechazo de 1000 [Hz] a 2000 [Hz], pero ahora buscaremos obtener los parámetros de la función *signal.tf2sos()*, tal que:

```

1  import numpy as np
2  from scipy import signal
3  import matplotlib.pyplot as plt
4
5  ## Ancho de Banda a Rechazar:
6  f1 = 1000                                # Frecuencia Inferior: 1000 [Hz]
7  f2 = 2000                                # Frecuencia Superior: 2000 [Hz]
8  Wbw = [2*np.pi*f1, 2*np.pi*f2]
9
10 ## Aproximación de un filtro rechaza banda, analógico y de segundo orden, con un
11 ## ripple en la banda de rechazo de 1 [dB] que sigue las frecuencias listadas
12 ## anteriormente.
13 num, den = signal.cheby1(N = 2,           # Filtro de 2do Orden
14                           rp = 1,         # Ripple de 1 [dB]
15                           Wn = Wbw,       # Frecuencias a Rechazar
16                           btype = 'bandstop', # Filtro rechaza banda
17                           analog = True,  # Filtro analógico
18                           output = 'ba')  # Salida del tipo: F(s) = num/den
19
20 ## Impresión de los parámetros obtenidos:
21 print("Numerador:", num)
22 print("Denominador:", den)
23

```



```

24  ## Descomposición en ceros, polos y constante
25  z, p, k = signal.tf2zpk(num, den)
26  print("Ceros: ", z)
27  print("Polos: ", p)
28  print("Constante: ", k)
29
30  ## Descomposición en biquadráticas mediante sos
31  sos = signal.tf2sos(num,den)
32  print("Coeficientes del SOS:", sos)

```

Al compilar este código, obtendremos la siguiente salida por consola:

```

Numerador: [8.91250938e-01 0.00000000e+00 1.40740707e+08 0.00000000e+00
5.55622040e+15]
Denominador: [1.00000000e+00 6.25596698e+03 1.93721424e+08 4.93951354e+11
6.23418183e+15]
Ceros: [-1.21894118e-07+8885.76595704j -1.21894118e-07-8885.76595704j
1.21893208e-07+8885.76579559j 1.21893208e-07-8885.76579559j]
Polos: [-2001.31728028+11672.49603722j -2001.31728028-11672.49603722j
-1126.66620836 +6571.17538633j -1126.66620836 -6571.17538633j]
Constante: 0.8912509381337455
Coeficientes del SOS: [[ 8.91250938e-01 2.17276494e-07 7.03703547e+07 1.00000000e+00
4.00263456e+03 1.40252435e+08]
[ 1.00000000e+00 -2.43786417e-07 7.89568338e+07 1.00000000e+00
2.25333242e+03 4.44497227e+07]]

```

Figura 3: Salida por consola del Ejemplo 2.

A este código, se le han incluido el uso de la función *signal.tf2zpk()* la cual nos entrega los ceros, polos y constante, y además se incluyó el mencionado uso de la función *signal.tf2sos()* para obtener los coeficientes de la función biquadrática. Se resume que los datos obtenidos y las funciones se pueden listar a continuación:

- Numerador: $[8,91x10^{-1}s^4 + 0s^3 + 1,40x10^8s^2 + 0s + 5,55x10^{15}]$
- Denominador: $[s^4 + 6,25x10^3s^3 + 1,93x10^8s^2 + 4,93x10^{11}s + 6,23x10^{15}]$
- Función de transferencia del filtro: Se presenta entonces de acuerdo a los dos items anteriores a la función del filtro:

$$T(s) = \frac{8,91x10^{-1}s^4 + 0s^3 + 1,40x10^8s^2 + 0s + 5,55x10^{15}}{s^4 + 6,25x10^3s^3 + 1,93x10^8s^2 + 4,93x10^{11}s + 6,23x10^{15}} \quad (4)$$

- Ceros: $z_{1,2} = -1,21894x10^{-7} \pm j8885,76595$; $z_{3,4} = 1,21893x10^{-7} \pm j8885,76579$
- Polos: $p_{1,2} = -2001,31 \pm j11672,49$; $p_{3,4} = -1126,66 \pm j6571,17$
- Constante: $K = 0,891250$

- Bicuadráticas: Se presenta las dos ecuaciones bicuadráticas que compondrán a este filtro:

$$T_1(s) = \frac{8,91x10^{-1}s^2 + 2,17x10^{-7}s + 7,03x10^{-7}}{s^2 + 4x10^3s + 1,4x10^8} \quad (5)$$

$$T_2(s) = \frac{s^2 - 2,43x10^{-7} + 7,89x10^7}{s^2 + 2,25x10^3 + 4,44x10^7} \quad (6)$$

2.2.3. Gráficas en Python

Si bien la descripción general de obtener un filtro y separarlo en distintas funciones bicuadráticas para luego poder sintetizarlo en un filtro activo siguiendo la bibliografía ya fue expuesto en los apartados anteriores, en esta sección se busca introducir el concepto y la habilidad de poder diseñar gráficas de bode para nuestro filtro, las cuales nos permitirán analizar el sistema desde un aspecto mas visual que simplemente analizando números.

En nuestro contexto resultan útiles las gráficas de bode y de respuesta en frecuencia para verificar que las frecuencias de corte sean las indicadas en el diagrama de módulo, y de los desfases en el tiempo que le ocurren a las señales de entrada al entrar al filtro en el diagrama de fase. Además, aquí podemos analizar la estabilidad del amplificador operacional, ya que este al no ser ideal, tiene en su interior una respuesta en frecuencia propia que puede alterar el comportamiento del filtro.

Para esto haremos uso de la interfaz *pyplot* dentro de la librería *matplotlib* la cual trae multitud de herramientas gráficas tales como diagramas de bode, respuesta en frecuencia, respuesta al escalón o inclusive graficar funciones, tal así como se realizo para la Tarea Complementaria N°1 donde se obtuvieron funciones y luego se realizaron gráficos de estas funciones

Para ello, se utiliza la función *signal.bode()* la cual toma como parámetros una función de transferencia, y devuelve un array de 3 elementos que representan la frecuencia (en radianes sobre segundo), la magnitud y la fase para poder armar los diagramas de bode.

Veamos entonces como implementar estos códigos para diseñar ahora un filtro en el mismo rango de frecuencias que para el ejemplo 1 y 2, pero ahora el filtro será un filtro pasa banda y estará aproximado por Butterworth en lugar de Chevyshev I. Se presenta a continuación el código utilizado:

```

1 import numpy as np
2 from scipy import signal
3 import matplotlib.pyplot as plt
4
5 ## Ancho de Banda a Aceptar:
6 f1 = 1000                                # Frecuencia Inferior: 1000 [Hz]
7 f2 = 2000                                # Frecuencia Superior: 2000 [Hz]
8 Wbw = [2*np.pi*f1, 2*np.pi*f2]
```

```

9
10 ## Aproximación de un filtro pasa banda, analógico y de segundo orden que sigue
11 ## las frecuencias listadas anteriormente.
12 num, den = signal.butter(N = 2,                                # Filtro de 2do Orden
13                           Wn = Wbw,                            # Frecuencias a Rechazar
14                           btype = 'bandpass',                  # Filtro pasa banda
15                           analog = True,                       # Filtro analógico
16                           output = 'ba')                       # Salida del tipo:  $F(s) = \text{num}/\text{den}$ 
17
18
19 ## Impresión de los parámetros obtenidos:
20 print("Numerador:", num)
21 print("Denominador:", den)
22
23 ## Descomposición en ceros, polos y constante
24 z, p, k = signal.tf2zpk(num, den)
25 print("Ceros: ", z)                                           # [0 ; 0]
26 print("Polos: ", p)                                           # [-2775.72 +- j11124.47 ;
27                                                         # -1667.15 +- j6681.59]
28 print("Constante: ", k)                                       # [39478417.60]
29
30 ## Descomposición en bicuadráticas mediante sos
31 sos = signal.tf2sos(num,den)                                    # [[3.94784176e+07 0 0 ;
32                                                         # 1 5.55144899e+03 1.31458664e+08]
33 print("Coeficientes del SOS:", sos)                            # [1 0 0 ;
34                                                         # 1 3.33431689e+03 4.74231339e+07]]
35
36 ## Diagrama de Bode de cada bicuadrática:
37 plt.figure()
38 for i in range(len(sos)):
39     w, mag, _ = signal.bode((sos[i, :3], sos[i, 3:]))
40     plt.semilogx(w/(2*np.pi), mag, label=f'Etapa {i+1}')
41 plt.xlabel('Frecuencia [Hz]')
42 plt.ylabel('Magnitud [dB]')
43 plt.title('Respuesta individual de cada biquad')
44 plt.grid(True)
45 plt.legend()
46 plt.show()
47

```

```

48  ## Respuesta en Frecuencia:
49  w, mag, phase = signal.bode((num, den))
50
51  # Gráfico de Magnitud
52  plt.figure()
53  plt.semilogx(w / (2*np.pi), mag)
54  plt.xlabel('Frecuencia [Hz]')
55  plt.ylabel('Magnitud [dB]')
56  plt.title('Diagrama de Bode - Módulo')
57  plt.grid(True)
58
59  # Gráfico de Fase
60  plt.figure()
61  plt.semilogx(w / (2*np.pi), phase)
62  plt.xlabel('Frecuencia [Hz]')
63  plt.ylabel('Fase [°]')
64  plt.title('Diagrama de Bode - Fase')
65  plt.grid(True)
66  plt.show()

```

De este código, se deducen los siguientes diagramas de bode de módulo y fase para el filtro:

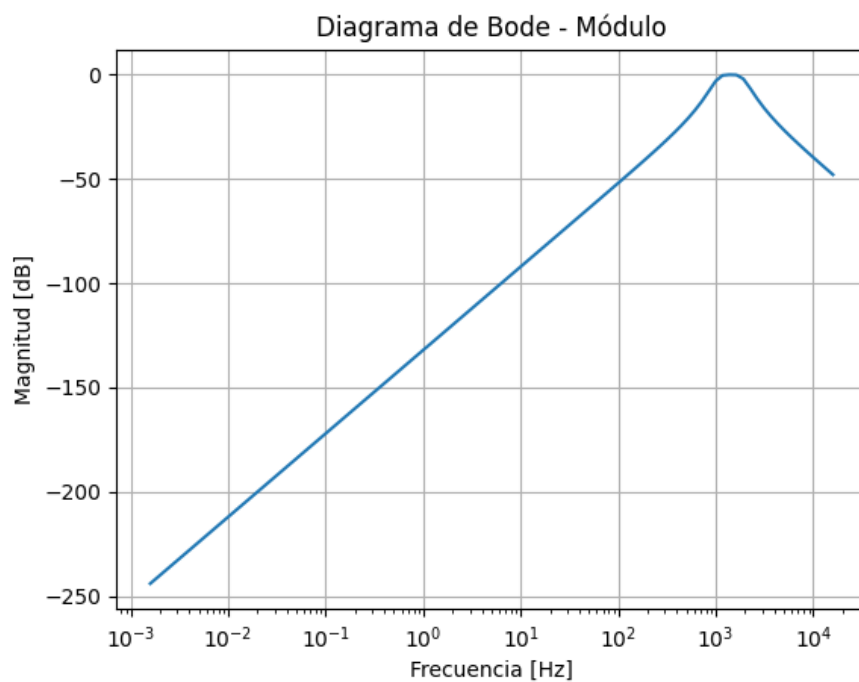


Figura 4: Diagrama de Bode - Módulo del Ejemplo 3 del Filtro Pasa Banda

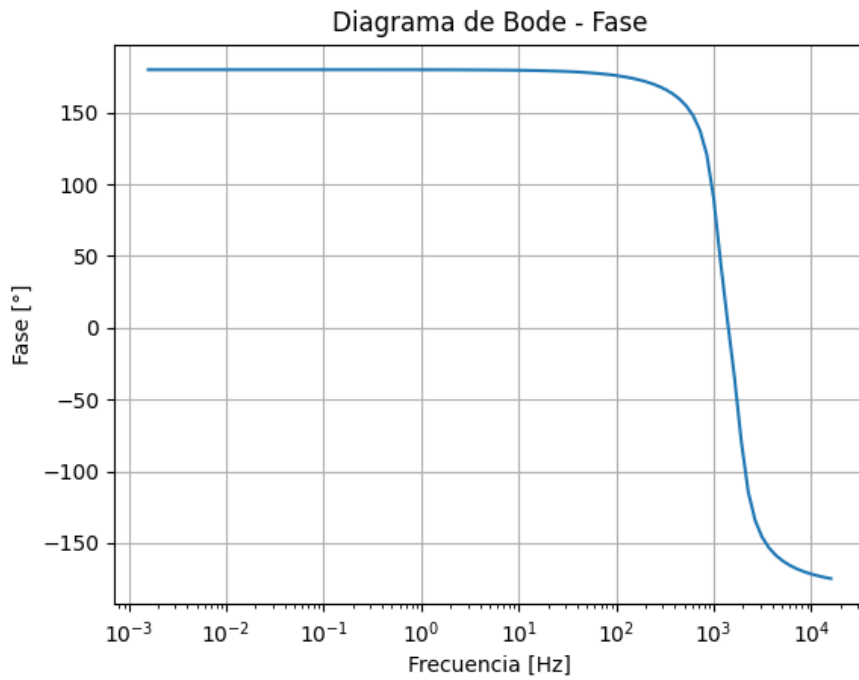


Figura 5: Diagrama de Bode - Fase del Ejemplo 3 del Filtro Pasa Banda

Por último, se presenta la respuesta en frecuencia individual de cada una de las bicuadráticas en su diagrama de bode de módulo:

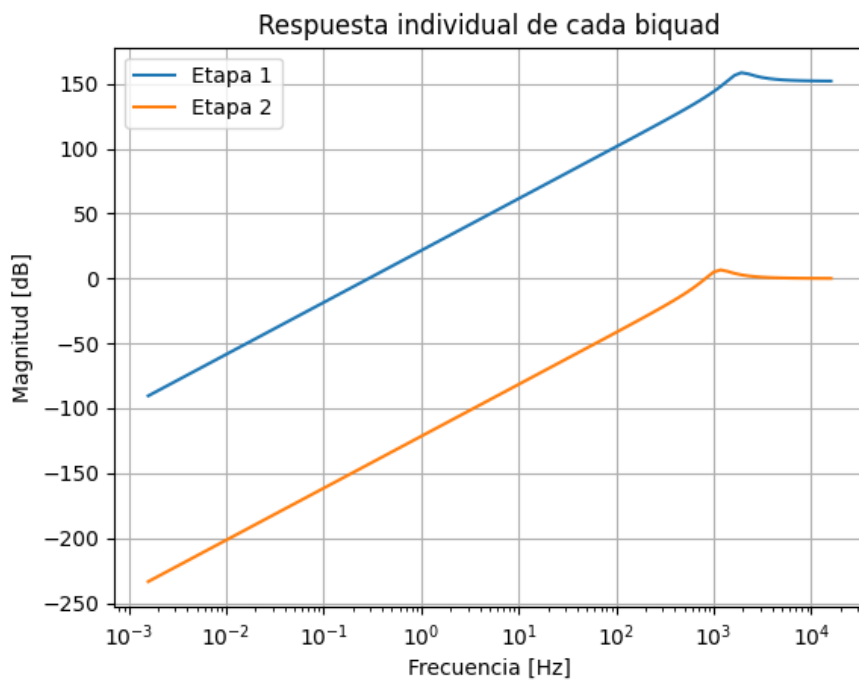


Figura 6: Diagrama de Bode - Módulo del Ejemplo 3 de cada bicuadrática

Se observa aquí la naturaleza propia del algoritmo, el cual genera 2 filtros pasa bajos tal como se espera, ya que si bien se eligió un filtro pasa banda, esta librería parte de un

filtro prototipo pasa bajo el cual luego le realiza una transformación para obtener el filtro deseado. La multiplicación de ambas etapas, nos dará el filtro pasa banda deseado, el cual seguirá el diagrama de bode presentado en la Figura 4.

3. Conclusión

A lo largo de este informe, se realizó una detallada explicación sobre el funcionamiento de las librerías de Python dedicadas a la obtención de funciones de transferencia de filtros dadas ciertas especificaciones del mismo. También se realizaron múltiples ejemplos de diseño para entender como es que funcionan y como realizar este proceso utilizando Python con todos los algoritmos que incluye esta librería.

4. Bibliografía

- Principles of Active Network Synthesis and Design - G. Daryanani
- Repositorio de GitHub con los códigos y demás archivos de este informe
- Documentación del módulo signal de la librería SciPy
- Documentación del módulo pyplot de la librería matplotlib

5. Anexo

- Anexo 1: Extracto de la función `signal.iirfilter()`:

```
1 def iirfilter(N, Wn, rp=None, rs=None, btype='band', analog=False,
2             ftype='butter', output='ba', fs=None):
3     """
4     IIR digital and analog filter design given order and critical points.
5
6     Design an Nth-order digital or analog filter and return the filter
7     coefficients.
8
9     Parameters
10    -----
11    N : int
12        The order of the filter.
13    Wn : array_like
14        A scalar or length-2 sequence giving the critical frequencies.
15
```

16 For digital filters, `Wn` are in the same units as `fs`. By default,
17 `fs` is 2 half-cycles/sample, so these are normalized from 0 to 1,
18 where 1 is the Nyquist frequency. (`Wn` is thus in
19 half-cycles / sample.)

20

21 For analog filters, `Wn` is an angular frequency (e.g., rad/s).

22

23 When `Wn` is a length-2 sequence, `Wn[0]` must be less than `Wn[1]`.

24 `rp` : float, optional
25 For Chebyshev and elliptic filters, provides the maximum ripple
26 in the passband. (dB)

27 `rs` : float, optional
28 For Chebyshev and elliptic filters, provides the minimum attenuation
29 in the stop band. (dB)

30 `btype` : {'bandpass', 'lowpass', 'highpass', 'bandstop'}, optional
31 The type of filter. Default is 'bandpass'.

32 `analog` : bool, optional
33 When True, return an analog filter, otherwise a digital filter is
34 returned.

35 `ftype` : str, optional
36 The type of IIR filter to design:

37

38 - Butterworth : 'butter'
39 - Chebyshev I : 'cheby1'
40 - Chebyshev II : 'cheby2'
41 - Cauer/elliptic: 'ellip'
42 - Bessel/Thomson: 'bessel'

43

44 `output` : {'ba', 'zpk', 'sos'}, optional
45 Filter form of the output:

46

47 - second-order sections (recommended): 'sos'
48 - numerator/denominator (default) : 'ba'
49 - pole-zero : 'zpk'

50

51 In general the second-order sections ('sos') form is
52 recommended because inferring the coefficients for the
53 numerator/denominator form ('ba') suffers from numerical
54 instabilities. For reasons of backward compatibility the default

55 *form is the numerator/denominator form ('ba'), where the 'b'*
 56 *and the 'a' in 'ba' refer to the commonly used names of the*
 57 *coefficients used.*

58

59 *Note: Using the second-order sections form ('sos') is sometimes*
 60 *associated with additional computational costs: for*
 61 *data-intense use cases it is therefore recommended to also*
 62 *investigate the numerator/denominator form ('ba').*

63

64 *fs : float, optional*
 65 *The sampling frequency of the digital system.*

66

67 *.. versionadded:: 1.2.0*

68

69 *Returns*
 70 *-----*

71 *b, a : ndarray, ndarray*
 72 *Numerator ('b') and denominator ('a') polynomials of the IIR filter.*
 73 *Only returned if ``output='ba'``.*

74 *z, p, k : ndarray, ndarray, float*
 75 *Zeros, poles, and system gain of the IIR filter transfer*
 76 *function. Only returned if ``output='zpk'``.*

77 *sos : ndarray*
 78 *Second-order sections representation of the IIR filter.*
 79 *Only returned if ``output='sos'``.*

80

81 *See Also*
 82 *-----*

83 *butter : Filter design using order and critical points*
 84 *cheby1, cheby2, ellip, bessel*
 85 *buttord : Find order and critical points from passband and stopband spec*
 86 *cheblord, cheb2ord, ellipord*
 87 *iirdesign : General filter design using passband and stopband spec*

88

89 *Notes*
 90 *-----*

91 *The ``'sos'`` output parameter was added in 0.16.0.*

92

93 *Examples*


```

94  -----
95  Generate a 17th-order Chebyshev II analog bandpass filter from 50 Hz to
96  200 Hz and plot the frequency response:
97
98  >>> import numpy as np
99  >>> from scipy import signal
100  >>> import matplotlib.pyplot as plt
101
102  >>> b, a = signal.iirfilter(17, [2*np.pi*50, 2*np.pi*200], rs=60,
103  ...                               btype='band', analog=True, ftype='cheby2')
104  >>> w, h = signal.freqs(b, a, 1000)
105  >>> fig = plt.figure()
106  >>> ax = fig.add_subplot(1, 1, 1)
107  >>> ax.semilogx(w / (2*np.pi), 20 * np.log10(np.maximum(abs(h), 1e-5)))
108  >>> ax.set_title('Chebyshev Type II bandpass frequency response')
109  >>> ax.set_xlabel('Frequency [Hz]')
110  >>> ax.set_ylabel('Amplitude [dB]')
111  >>> ax.axis((10, 1000, -100, 10))
112  >>> ax.grid(which='both', axis='both')
113  >>> plt.show()
114
115  Create a digital filter with the same properties, in a system with
116  sampling rate of 2000 Hz, and plot the frequency response. (Second-order
117  sections implementation is required to ensure stability of a filter of
118  this order):
119
120  >>> sos = signal.iirfilter(17, [50, 200], rs=60, btype='band',
121  ...                               analog=False, ftype='cheby2', fs=2000,
122  ...                               output='sos')
123  >>> w, h = signal.freqz_sos(sos, 2000, fs=2000)
124  >>> fig = plt.figure()
125  >>> ax = fig.add_subplot(1, 1, 1)
126  >>> ax.semilogx(w, 20 * np.log10(np.maximum(abs(h), 1e-5)))
127  >>> ax.set_title('Chebyshev Type II bandpass frequency response')
128  >>> ax.set_xlabel('Frequency [Hz]')
129  >>> ax.set_ylabel('Amplitude [dB]')
130  >>> ax.axis((10, 1000, -100, 10))
131  >>> ax.grid(which='both', axis='both')
132  >>> plt.show()

```

```

133
134     """
135     fs = _validate_fs(fs, allow_none=True)
136     ftype, btype, output = (x.lower() for x in (ftype, btype, output))
137     Wn = asarray(Wn)
138     if fs is not None:
139         if analog:
140             raise ValueError("fs cannot be specified for an analog filter")
141         Wn = Wn / (fs/2)
142
143     if np.any(Wn <= 0):
144         raise ValueError("filter critical frequencies must be greater than 0")
145
146     if Wn.size > 1 and not Wn[0] < Wn[1]:
147         raise ValueError("Wn[0] must be less than Wn[1]")
148
149     try:
150         btype = band_dict[btype]
151     except KeyError as e:
152         raise ValueError(f"'{btype}' is an invalid bandtype for filter.") from e
153
154     try:
155         typefunc = filter_dict[ftype][0]
156     except KeyError as e:
157         raise ValueError(f"'{ftype}' is not a valid basic IIR filter.") from e
158
159     if output not in ['ba', 'zpk', 'sos']:
160         raise ValueError(f"'{output}' is not a valid output form.")
161
162     if rp is not None and rp < 0:
163         raise ValueError("passband ripple (rp) must be positive")
164
165     if rs is not None and rs < 0:
166         raise ValueError("stopband attenuation (rs) must be positive")
167
168     # Get analog lowpass prototype
169     if typefunc == buttap:
170         z, p, k = typefunc(N)
171     elif typefunc == besslap:

```

```

172         z, p, k = typefunc(N, norm=bessel_norms[ftype])
173     elif typefunc == cheblap:
174         if rp is None:
175             raise ValueError("passband ripple (rp) must be provided to "
176                               "design a Chebyshev I filter.")
177         z, p, k = typefunc(N, rp)
178     elif typefunc == cheb2ap:
179         if rs is None:
180             raise ValueError("stopband attenuation (rs) must be provided to "
181                               "design an Chebyshev II filter.")
182         z, p, k = typefunc(N, rs)
183     elif typefunc == ellipap:
184         if rs is None or rp is None:
185             raise ValueError("Both rp and rs must be provided to design an "
186                               "elliptic filter.")
187         z, p, k = typefunc(N, rp, rs)
188     else:
189         raise NotImplementedError(f"'{ftype}' not implemented in iirfilter.")
190
191     # Pre-warp frequencies for digital filter design
192     if not analog:
193         if np.any(Wn <= 0) or np.any(Wn >= 1):
194             if fs is not None:
195                 raise ValueError("Digital filter critical frequencies must "
196                                   f"be 0 < Wn < fs/2 (fs={fs} -> fs/2={fs/2})")
197             raise ValueError("Digital filter critical frequencies "
198                               "must be 0 < Wn < 1")
199         fs = 2.0
200         warped = 2 * fs * tan(pi * Wn / fs)
201     else:
202         warped = Wn
203
204     # transform to lowpass, bandpass, highpass, or bandstop
205     if btype in ('lowpass', 'highpass'):
206         if np.size(Wn) != 1:
207             raise ValueError('Must specify a single critical frequency Wn '
208                               'for lowpass or highpass filter')
209
210         if btype == 'lowpass':

```

```

211         z, p, k = lp2lp_zpk(z, p, k, wo=warped)
212     elif btype == 'highpass':
213         z, p, k = lp2hp_zpk(z, p, k, wo=warped)
214 elif btype in ('bandpass', 'bandstop'):
215     try:
216         bw = warped[1] - warped[0]
217         wo = sqrt(warped[0] * warped[1])
218     except IndexError as e:
219         raise ValueError('Wn must specify start and stop frequencies for '
220                           'bandpass or bandstop filter') from e
221
222     if btype == 'bandpass':
223         z, p, k = lp2bp_zpk(z, p, k, wo=wo, bw=bw)
224     elif btype == 'bandstop':
225         z, p, k = lp2bs_zpk(z, p, k, wo=wo, bw=bw)
226 else:
227     raise NotImplementedError(f"'{btype}' not implemented in iirfilter.")
228
229     # Find discrete equivalent if necessary
230     if not analog:
231         z, p, k = bilinear_zpk(z, p, k, fs=fs)
232
233     # Transform to proper out type (pole-zero, state-space, numer-denom)
234     if output == 'zpk':
235         return z, p, k
236     elif output == 'ba':
237         return zpk2tf(z, p, k)
238     elif output == 'sos':
239         return zpk2sos(z, p, k, analog=analog)

```

- Anexo 2: Extracto de la función *signal.zpk2sos()*

```

1 def zpk2sos(z, p, k, pairing=None, *, analog=False):
2     """Return second-order sections from zeros, poles, and gain of a system
3
4     Parameters
5     -----
6     z : array_like
7         Zeros of the transfer function.
8     p : array_like
9         Poles of the transfer function.

```

```

10     k : float
11         System gain.
12     pairing : {None, 'nearest', 'keep_odd', 'minimal'}, optional
13         The method to use to combine pairs of poles and zeros into sections.
14         If analog is False and pairing is None, pairing is set to 'nearest';
15         if analog is True, pairing must be 'minimal', and is set to that if
16         it is None.
17     analog : bool, optional
18         If True, system is analog, otherwise discrete.
19
20     .. versionadded:: 1.8.0
21
22     Returns
23     -----
24     sos : ndarray
25         Array of second-order filter coefficients, with shape
26         ``(n_sections, 6)``. See `sosfilt` for the SOS filter format
27         specification.
28
29     See Also
30     -----
31     sosfilt
32
33     Notes
34     -----
35     The algorithm used to convert ZPK to SOS format is designed to
36     minimize errors due to numerical precision issues. The pairing
37     algorithm attempts to minimize the peak gain of each biquadratic
38     section. This is done by pairing poles with the nearest zeros, starting
39     with the poles closest to the unit circle for discrete-time systems, and
40     poles closest to the imaginary axis for continuous-time systems.
41
42     ``pairing='minimal'`` outputs may not be suitable for `sosfilt`,
43     and ``analog=True`` outputs will never be suitable for `sosfilt`.
44
45     *Algorithms*
46
47     The steps in the ``pairing='nearest'``, ``pairing='keep_odd'``,
48     and ``pairing='minimal'`` algorithms are mostly shared. The

```

49 *``'nearest'`` algorithm attempts to minimize the peak gain, while*
50 *``'keep\_odd'`` minimizes peak gain under the constraint that*
51 *odd-order systems should retain one section as first order.*
52 *``'minimal'`` is similar to ``'keep\_odd'``, but no additional*
53 *poles or zeros are introduced*

54
55 *The algorithm steps are as follows:*

56
57 *As a pre-processing step for ``pairing='nearest'``,*
58 *``pairing='keep\_odd'``, add poles or zeros to the origin as*
59 *necessary to obtain the same number of poles and zeros for*
60 *pairing. If ``pairing == 'nearest'`` and there are an odd number*
61 *of poles, add an additional pole and a zero at the origin.*

62
63 *The following steps are then iterated over until no more poles or*
64 *zeros remain:*

- 65
66 1. *Take the (next remaining) pole (complex or real) closest to the*
67 *unit circle (or imaginary axis, for ``analog=True``) to*
68 *begin a new filter section.*
69
70 2. *If the pole is real and there are no other remaining real poles [#]_,*
71 *add the closest real zero to the section and leave it as a first*
72 *order section. Note that after this step we are guaranteed to be*
73 *left with an even number of real poles, complex poles, real zeros,*
74 *and complex zeros for subsequent pairing iterations.*

75
76 3. *Else:*

- 77
78 1. *If the pole is complex and the zero is the only remaining real*
79 *zero*, then pair the pole with the *next* closest zero*
80 *(guaranteed to be complex). This is necessary to ensure that*
81 *there will be a real zero remaining to eventually create a*
82 *first-order section (thus keeping the odd order).*

- 83
84 2. *Else pair the pole with the closest remaining zero (complex or*
85 *real).*

- 86
87 3. *Proceed to complete the second-order section by adding another*

```

88         pole and zero to the current pole and zero in the section:
89
90         1. If the current pole and zero are both complex, add their
91            conjugates.
92
93         2. Else if the pole is complex and the zero is real, add the
94            conjugate pole and the next closest real zero.
95
96         3. Else if the pole is real and the zero is complex, add the
97            conjugate zero and the real pole closest to those zeros.
98
99         4. Else (we must have a real pole and real zero) add the next
100            real pole closest to the unit circle, and then add the real
101            zero closest to that pole.
102
103     .. [#] This conditional can only be met for specific odd-order inputs
104        with the ``pairing = 'keep_odd'`` or ``'minimal'`` methods.
105
106     .. versionadded:: 0.16.0
107
108     Examples
109     -----
110
111     Design a 6th order low-pass elliptic digital filter for a system with a
112     sampling rate of 8000 Hz that has a pass-band corner frequency of
113     1000 Hz. The ripple in the pass-band should not exceed 0.087 dB, and
114     the attenuation in the stop-band should be at least 90 dB.
115
116     In the following call to ellip, we could use output='sos',
117     but for this example, we'll use output='zpk', and then convert
118     to SOS format with zpk2sos:
119
120     >>> from scipy import signal
121     >>> import numpy as np
122     >>> z, p, k = signal.ellip(6, 0.087, 90, 1000/(0.5*8000), output='zpk')
123
124     Now convert to SOS format.
125
126     >>> sos = signal.zpk2sos(z, p, k)

```

127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165

The coefficients of the numerators of the sections:

```
>>> sos[:, :3]
array([[0.0014152 , 0.00248677, 0.0014152 ],
       [1.          , 0.72976874, 1.          ],
       [1.          , 0.17607852, 1.          ]])
```

The symmetry in the coefficients occurs because all the zeros are on the unit circle.

The coefficients of the denominators of the sections:

```
>>> sos[:, 3:]
array([[ 1.          , -1.32544025,  0.46989976],
       [ 1.          , -1.26118294,  0.62625924],
       [ 1.          , -1.2570723 ,  0.8619958 ]])
```

The next example shows the effect of the `pairing` option. We have a system with three poles and three zeros, so the SOS array will have shape (2, 6). The means there is, in effect, an extra pole and an extra zero at the origin in the SOS representation.

```
>>> z1 = np.array([-1, -0.5-0.5j, -0.5+0.5j])
>>> p1 = np.array([0.75, 0.8+0.1j, 0.8-0.1j])
```

With ``pairing='nearest'`` (the default), we obtain

```
>>> signal.zpk2sos(z1, p1, 1)
array([[ 1. ,  1. ,  0.5 ,  1. , -0.75,  0. ],
       [ 1. ,  1. ,  0. ,  1. , -1.6 ,  0.65]])
```

The first section has the zeros $\{-0.5-0.05j, -0.5+0.5j\}$ and the poles $\{0, 0.75\}$, and the second section has the zeros $\{-1, 0\}$ and poles $\{0.8+0.1j, 0.8-0.1j\}$. Note that the extra pole and zero at the origin have been assigned to different sections.

With ``pairing='keep\_odd'``, we obtain:


```

166 >>> signal.zpk2sos(z1, p1, 1, pairing='keep_odd')
167 array([[ 1.   ,  1.   ,  0.   ,  1.   , -0.75,  0.   ],
168        [ 1.   ,  1.   ,  0.5  ,  1.   , -1.6  ,  0.65]])
169
170 The extra pole and zero at the origin are in the same section.
171 The first section is, in effect, a first-order section.
172
173 With ``pairing='minimal'``, the first-order section doesn't have
174 the extra pole and zero at the origin:
175
176 >>> signal.zpk2sos(z1, p1, 1, pairing='minimal')
177 array([[ 0.   ,  1.   ,  1.   ,  0.   ,  1.   , -0.75],
178        [ 1.   ,  1.   ,  0.5  ,  1.   , -1.6  ,  0.65]])
179
180 """
181 # TODO in the near future:
182 # 1. Add SOS capability to `filtfilt`, `freqz`, etc. somehow (#3259).
183 # 2. Make `decimate` use `sosfilt` instead of `lfilter`.
184 # 3. Make sosfilt automatically simplify sections to first order
185 #     when possible. Note this might make `sosfiltfilt` a bit harder (ICs).
186 # 4. Further optimizations of the section ordering / pole-zero pairing.
187 # See the wiki for other potential issues.
188
189 xp = array_namespace(z, p)
190
191 # convert to numpy, convert back on exit   XXX
192 z, p = map(np.asarray, (z, p))
193 k = np.asarray(k)
194
195 if pairing is None:
196     pairing = 'minimal' if analog else 'nearest'
197
198 valid_pairings = ['nearest', 'keep_odd', 'minimal']
199 if pairing not in valid_pairings:
200     raise ValueError(f'pairing must be one of {valid_pairings}, not {pairing}')
201
202 if analog and pairing != 'minimal':
203     raise ValueError('for analog zpk2sos conversion, '
204                       'pairing must be "minimal"')

```

```

205
206 if len(z) == len(p) == 0:
207     if not analog:
208         return xp.asarray(np.asarray([[k, 0., 0., 1., 0., 0.])))
209     else:
210         return xp.asarray(np.asarray([[0., 0., k, 0., 0., 1.])))
211
212 if pairing != 'minimal':
213     # ensure we have the same number of poles and zeros, and make copies
214     p = np.concatenate((p, np.zeros(max(len(z) - len(p), 0))))
215     z = np.concatenate((z, np.zeros(max(len(p) - len(z), 0))))
216     n_sections = (max(len(p), len(z)) + 1) // 2
217
218     if len(p) % 2 == 1 and pairing == 'nearest':
219         p = np.concatenate((p, [0.]))
220         z = np.concatenate((z, [0.]))
221         assert len(p) == len(z)
222 else:
223     if len(p) < len(z):
224         raise ValueError('for analog zpk2sos conversion, '
225                          'must have len(p)>=len(z)')
226
227     n_sections = (len(p) + 1) // 2
228
229     # Ensure we have complex conjugate pairs
230     # (note that _cplxreal only gives us one element of each complex pair):
231     z = np.concatenate(_cplxreal(z))
232     p = np.concatenate(_cplxreal(p))
233     if not np.isreal(k):
234         raise ValueError('k must be real')
235     k = k.real
236
237 if not analog:
238     # digital: "worst" is the closest to the unit circle
239     def idx_worst(p):
240         return np.argmin(np.abs(1 - np.abs(p)))
241 else:
242     # analog: "worst" is the closest to the imaginary axis
243     def idx_worst(p):

```

```

244         return np.argmin(np.abs(np.real(p)))
245
246     sos = np.zeros((n_sections, 6))
247
248     # Construct the system, reversing order so the "worst" are last
249     for si in range(n_sections-1, -1, -1):
250         # Select the next "worst" pole
251         p1_idx = idx_worst(p)
252         p1 = p[p1_idx]
253         p = np.delete(p, p1_idx)
254
255         # Pair that pole with a zero
256
257         if np.isreal(p1) and np.isreal(p).sum() == 0:
258             # Special case (1): last remaining real pole
259             if pairing != 'minimal':
260                 z1_idx = _nearest_real_complex_idx(z, p1, 'real')
261                 z1 = z[z1_idx]
262                 z = np.delete(z, z1_idx)
263                 sos[si] = _single_zpk sos([z1, 0], [p1, 0], 1)
264             elif len(z) > 0:
265                 z1_idx = _nearest_real_complex_idx(z, p1, 'real')
266                 z1 = z[z1_idx]
267                 z = np.delete(z, z1_idx)
268                 sos[si] = _single_zpk sos([z1], [p1], 1)
269             else:
270                 sos[si] = _single_zpk sos([], [p1], 1)
271
272         elif (len(p) + 1 == len(z)
273              and not np.isreal(p1)
274              and np.isreal(p).sum() == 1
275              and np.isreal(z).sum() == 1):
276
277             # Special case (2): there's one real pole and one real zero
278             # left, and an equal number of poles and zeros to pair up.
279             # We *must* pair with a complex zero
280
281             z1_idx = _nearest_real_complex_idx(z, p1, 'complex')
282             z1 = z[z1_idx]

```

```

283         z = np.delete(z, z1_idx)
284         sos[si] = _single_zpk sos([z1, z1.conj()], [p1, p1.conj()], 1)
285
286     else:
287         if np.isreal(p1):
288             prealidx = np.flatnonzero(np.isreal(p))
289             p2_idx = prealidx[idx_worst(p[prealidx])]
290             p2 = p[p2_idx]
291             p = np.delete(p, p2_idx)
292         else:
293             p2 = p1.conj()
294
295         # find closest zero
296         if len(z) > 0:
297             z1_idx = _nearest_real_complex_idx(z, p1, 'any')
298             z1 = z[z1_idx]
299             z = np.delete(z, z1_idx)
300
301         if not np.isreal(z1):
302             sos[si] = _single_zpk sos([z1, z1.conj()], [p1, p2], 1)
303         else:
304             if len(z) > 0:
305                 z2_idx = _nearest_real_complex_idx(z, p1, 'real')
306                 z2 = z[z2_idx]
307                 assert np.isreal(z2)
308                 z = np.delete(z, z2_idx)
309                 sos[si] = _single_zpk sos([z1, z2], [p1, p2], 1)
310             else:
311                 sos[si] = _single_zpk sos([z1], [p1, p2], 1)
312         else:
313             # no more zeros
314             sos[si] = _single_zpk sos([], [p1, p2], 1)
315
316     assert len(p) == len(z) == 0 # we've consumed all poles and zeros
317     del p, z
318
319     # put gain in first sos
320     sos[0][:3] *= k
321     return xp.asarray(sos)

```